JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface SecureDirectoryStream<T>

Type Parameters:

T - The type of element returned by the iterator

All Superinterfaces:

AutoCloseable, Closeable, DirectoryStream<T>, Iterable<T>

```
public interface SecureDirectoryStream<T>
extends DirectoryStream<T>
```

A DirectoryStream that defines operations on files that are located relative to an open directory. A SecureDirectoryStream is intended for use by sophisticated or security sensitive applications requiring to traverse file trees or otherwise operate on directories in a race-free manner. Race conditions can arise when a sequence of file operations cannot be carried out in isolation. Each of the file operations defined by this interface specify a relative path. All access to the file is relative to the open directory irrespective of if the directory is moved or replaced by an attacker while the directory is open. A SecureDirectoryStream may also be used as a virtual *working directory*.

A SecureDirectoryStream requires corresponding support from the underlying operating system. Where an implementation supports this features then the DirectoryStream returned by the newDirectoryStream method will be a SecureDirectoryStream and must be cast to that type in order to invoke the methods defined by this interface.

Since:

1.7

Nested Class Summary

Nested classes/interfaces declared in interface DirectoryStream
DirectoryStream.Filter<T>

Method Summary

All Methods	Instance Methods	
Modifier and Type	**Method**	**Description**
void	deleteDirectory(T path)	Deletes a directory.
void	deleteFile(T path)	Deletes a file.

<V extends FileAttributeView> V	**getFileAttributeView**(Class<V> type)	Returns a new file attribute view to access the file attributes of this directory.
<V extends FileAttributeView> V	**getFileAttributeView**(T path, Class<V> type, LinkOption... options)	Returns a new file attribute view to access the file attributes of a file in this directory.
void	**move**(T srcpath, SecureDirectoryStream<T> T targetpath)	Move a file from this directory to another directory.
SeekableByteChannel	**newByteChannel**(T path, Set<? extends OpenOption> options, FileAttribute<? >... attrs)	Opens or creates a file in this directory, returning a seekable byte channel to access the file.
SecureDirectoryStrea <T>	**newDirectoryStream**(T path, LinkOption... options)	Opens the directory identified by the given path, returning a SecureDirectoryStream to iterate over the entries in the directory.

Methods declared in interface Closeable

close

Methods declared in interface DirectoryStream

iterator

Methods declared in interface Iterable

`forEach`, `spliterator`

Method Details

newDirectoryStream

```
SecureDirectoryStream<T> newDirectoryStream(T path,
                                            LinkOption... options)
                                     throws IOException
```

Opens the directory identified by the given path, returning a `SecureDirectoryStream` to iterate over the entries in the directory.

This method works in exactly the manner specified by the `newDirectoryStream` method for the case that the `path` parameter is an `absolute` path. When the parameter is a relative path then the directory to open is relative to this open directory. The `NOFOLLOW_LINKS` option may be used to ensure that this method fails if the file is a symbolic link.

The new directory stream, once created, is not dependent upon the directory stream used to create it. Closing this directory stream has no effect upon newly created directory stream.

Parameters:

`path` - the path to the directory to open

`options` - options indicating how symbolic links are handled

Returns:

a new and open `SecureDirectoryStream` object

Throws:

`ClosedDirectoryStreamException` - if the directory stream is closed

`NotDirectoryException` - if the file could not otherwise be opened because it is not a directory *(optional specific exception)*

`IOException` - if an I/O error occurs

newByteChannel

```
SeekableByteChannel newByteChannel(T path,
                                   Set<? extends OpenOption> options,
                                   FileAttribute<?>... attrs)
                            throws IOException
```

Opens or creates a file in this directory, returning a seekable byte channel to access the file.

This method works in exactly the manner specified by the `Files.newByteChannel` method for the case that the `path` parameter is an `absolute` path. When the parameter is a relative path then the file to open or create is relative to this open directory. In addition to the options defined by the `Files.newByteChannel` method, the `NOFOLLOW_LINKS` option may be used to ensure that this method fails if the file is a symbolic link.

The channel, once created, is not dependent upon the directory stream used to create it. Closing this directory stream has no effect upon the channel.

Parameters:

`path` - the path of the file to open or create

`options` - options specifying how the file is opened

`attrs` - an optional list of attributes to set atomically when creating the file

Returns:

the seekable byte channel

Throws:

`ClosedDirectoryStreamException` - if the directory stream is closed

`IllegalArgumentException` - if the set contains an invalid combination of options

`UnsupportedOperationException` - if an unsupported open option is specified or the array contains attributes that cannot be set atomically when creating the file

`FileAlreadyExistsException` - if a file of that name already exists and the `CREATE_NEW` option is specified *(optional specific exception)*

`IOException` - if an I/O error occurs

deleteFile

```
void deleteFile(T path)
         throws IOException
```

Deletes a file.

Unlike the `delete()` method, this method does not first examine the file to determine if the file is a directory. Whether a directory is deleted by this method is system dependent and therefore not specified. If the file is a symbolic link, then the link itself, not the final target of the link, is deleted. When the parameter is a relative path then the file to delete is relative to this open directory.

Parameters:

`path` - the path of the file to delete

Throws:

`ClosedDirectoryStreamException` - if the directory stream is closed

`NoSuchFileException` - if the file does not exist *(optional specific exception)*

`IOException` - if an I/O error occurs

deleteDirectory

```
void deleteDirectory(T path)
              throws IOException
```

Deletes a directory.

Unlike the `delete()` method, this method does not first examine the file to determine if the file is a directory. Whether non-directories are deleted by this method is system dependent and therefore not specified. When the parameter is a relative path then the directory to delete is relative to this open directory.

Parameters:

`path` - the path of the directory to delete

Throws:

`ClosedDirectoryStreamException` - if the directory stream is closed

`NoSuchFileException` - if the directory does not exist *(optional specific exception)*

`DirectoryNotEmptyException` - if the directory could not otherwise be deleted because it is not empty *(optional specific exception)*

`IOException` - if an I/O error occurs

move

```
void move(T srcpath,
          SecureDirectoryStream<T> targetdir,
          T targetpath)
   throws IOException
```

Move a file from this directory to another directory.

This method works in a similar manner to `Files.move` when the `ATOMIC_MOVE` option is specified. That is, this method moves a file as an atomic file system operation. If the `srcpath` parameter is an `absolute` path then it locates the source file. If the parameter is a relative path then it is located relative to this open directory. If the `targetpath` parameter is absolute then it locates the target file (the `targetdir` parameter is ignored). If the parameter is a relative

path it is located relative to the open directory identified by the `targetdir` parameter, unless `targetdir` is `null`, in which case it is located relative to the current working directory. In all cases, if the target file exists then it is implementation specific if it is replaced or this method fails.

Parameters:

`srcpath` - the name of the file to move

`targetdir` - the destination directory; can be `null`

`targetpath` - the name to give the file in the destination directory

Throws:

`ClosedDirectoryStreamException` - if this or the target directory stream is closed

`FileAlreadyExistsException` - if the file already exists in the target directory and cannot be replaced *(optional specific exception)*

`AtomicMoveNotSupportedException` - if the file cannot be moved as an atomic file system operation

`IOException` - if an I/O error occurs

getFileAttributeView

```
<V extends FileAttributeView> V getFileAttributeView(Class<V> type)
```

Returns a new file attribute view to access the file attributes of this directory.

The resulting file attribute view can be used to read or update the attributes of this (open) directory. The `type` parameter specifies the type of the attribute view and the method returns an instance of that type if supported. Invoking this method to obtain a `BasicFileAttributeView` always returns an instance of that class that is bound to this open directory.

The state of resulting file attribute view is intimately connected to this directory stream. Once the directory stream is `closed`, then all methods to read or update attributes will throw `ClosedDirectoryStreamException`.

Type Parameters:

`V` - The `FileAttributeView` type

Parameters:

`type` - the `Class` object corresponding to the file attribute view

Returns:

a new file attribute view of the specified type bound to this directory stream, or `null` if the attribute view type is not available

getFileAttributeView

```
<V extends FileAttributeView> V getFileAttributeView
(T path,
 Class<V> type,
 LinkOption... options)
```

Returns a new file attribute view to access the file attributes of a file in this directory.

The resulting file attribute view can be used to read or update the attributes of file in this directory. The `type` parameter specifies the type of the attribute view and the method returns an instance of that type if supported. Invoking this method to obtain a `BasicFileAttributeView` always returns an instance of that class that is bound to the file in the directory.

The state of resulting file attribute view is intimately connected to this directory stream. Once the directory stream `closed`, then all methods to read or update attributes will throw `ClosedDirectoryStreamException`. The file is not required to exist at the time that the file attribute view is created but methods to read or update attributes of the file will fail when invoked and the file does not exist.

Type Parameters:

`V` - The `FileAttributeView` type

Parameters:

`path` - the path of the file

`type` - the `Class` object corresponding to the file attribute view

`options` - options indicating how symbolic links are handled

Returns:

a new file attribute view of the specified type bound to this directory stream, or `null` if the attribute view type is not available

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.